Technical Report: Network Traffic Forensic & Mobile Artifacts Extraction

Author: Le Chi Bach

Table of Contents

Q1. Network forensics from network capture 3
a. Wireshark Filter for Identifying Potential Attack Traffic 3
b. Identifying the Attacker's Host from Wireshark Filter Output 6
Q2: Mobile Data Acquisition and Analysis 7
Part 1: 7
a. Create an Android Virtual Device using Android Studio 7
b. Create Contacts and Messages 13
Part 2: 16
c. Data acquisition using ADB tool 16
d. Identify the locations and names of the SMS and Contact databases 19
e. Copy the databases to the computer. 21
f. Access permission issue 21
g. Retrieve the SMSs and the contact details. 22
h. Conclusion 23
References 24

Q1. Network forensics from network capture

a. Wireshark Filter for Identifying Potential Attack Traffic

In this task, we are a forensic analyst and have been assigned to investigate a case from one of our clients. It is reported that our client's server had been under a network-based Bot attack (possibly HTTP Flood and Distributed Denial of Service or DDoS), and that the server resources were overwhelmed by requests from various attack machines. It is suspected that the Bot, which is allegedly behind the attack, dynamically changed its user agents, HTTP method types and add a suffix to the end of URLs to penetrate Content Delivery Network (CDN) systems' security. The attack had made the server unreachable to legitimate users.

As suggested by the requirements, the possibility of DDoS will be examined. Therefore, the filter used for this case will be **tcp.flags.syn == 1 and tcp.flags.ack == 0 or http.request.method == "POST" or http.request.method == "GET"**. Applying to the filter, Wireshark will return the list of TCP packets with a syn flag with no corresponding ack flag, as well as the number of HTTP packets with POST and GET request, hoping to find a sign of HTTP flood and SYN flood. To make it easier to visualize, the filter will be broken down into 2 parts: **tcp.flags.syn == 1 and tcp.flags.ack == 0** and **http.request.method == "POST" or http.request.method == "GET".**

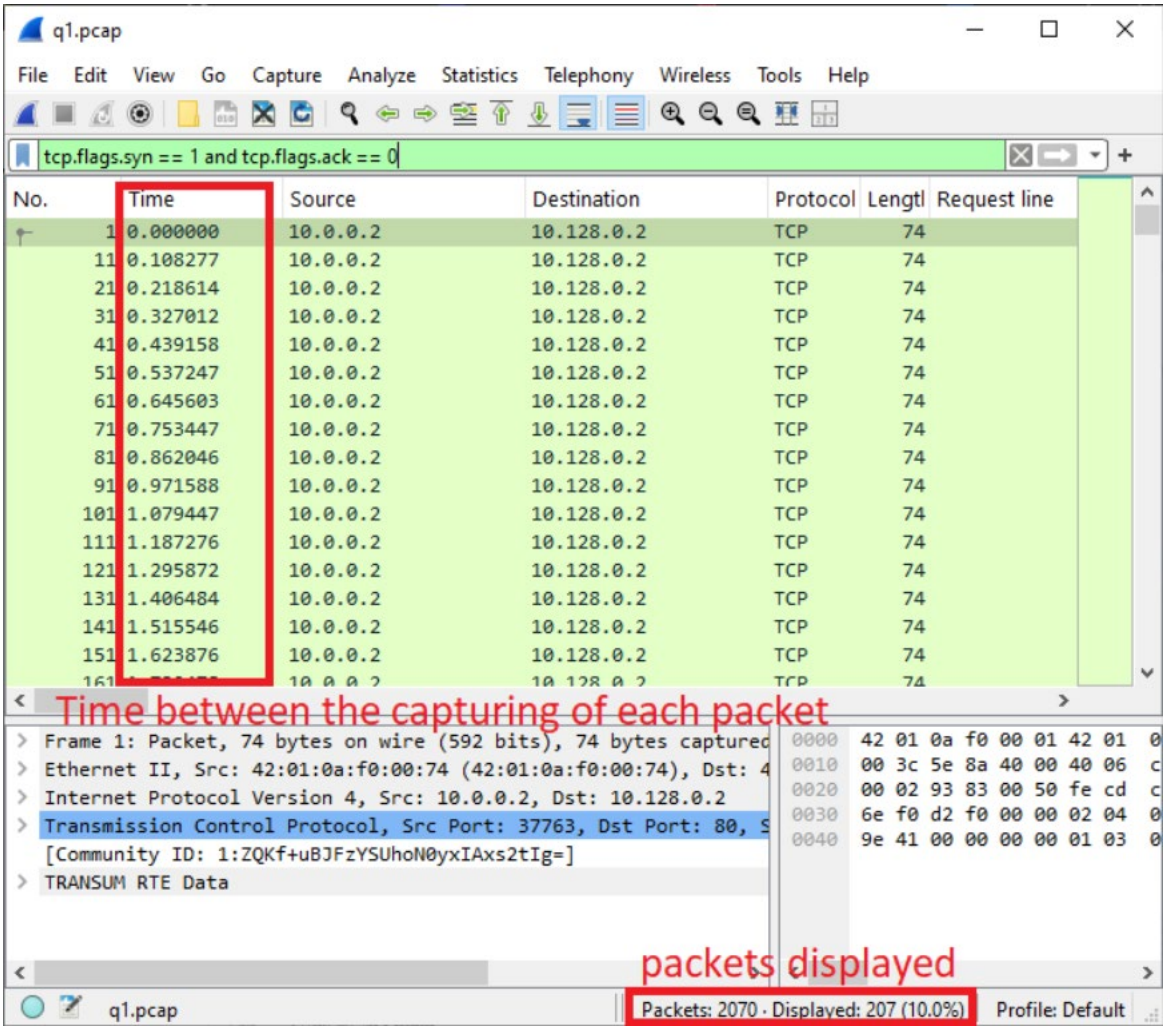


Figure 1: Wireshark filter result for tcp.flags.syn == 1 and tcp.flags.ack == 0.

As shown in Figure 1, 207 packets matched the first part of the filter. TCP protocol uses a **3-way-handshake** process to ensure a reliable client-server connection. The client side sends a **SYN** message to the server; the server will receive the **SYN number** and reply with its own **SYN** and an **ACK** message. The client will then receive the **SYN-ACK** message and reply with an **ACK**, which the server is expecting to receive. As seen in the image, there are a lot of packets with a **SYN** flag but without an **ACK** flag[1]. The client side didn't send the **ACK** message, so the server must leave an open port on timeout to wait for that response. We can also see that a number of these packets were sent within a second. This process will occupy all available ports of the server, leaving it unavailable for legitimate users. The attackers have used **SYN flood methods** to create the DDoS attack [2].

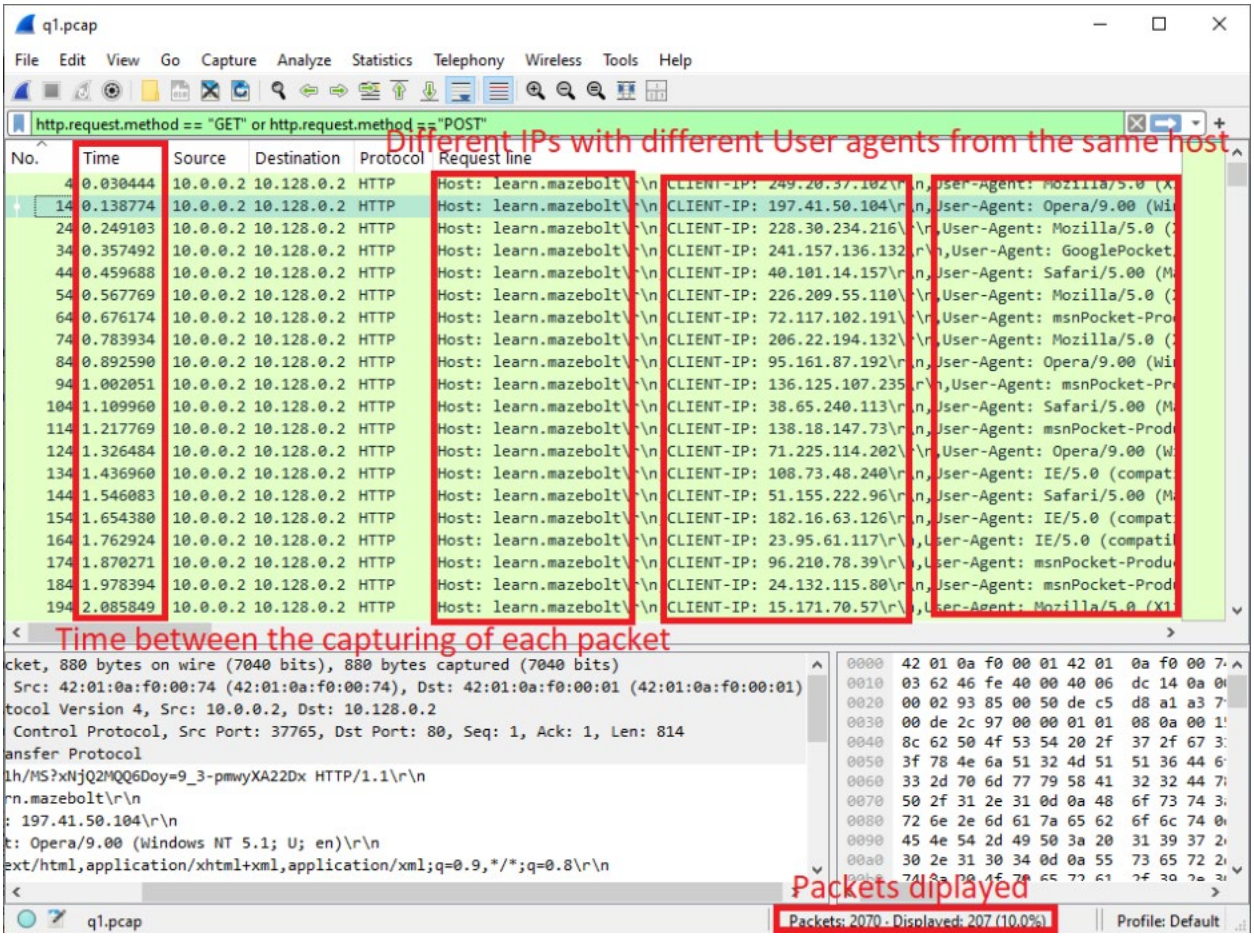


Figure 2: Wireshark filter result for http.request.method == "POST" or http.request.method == "GET"

As shown in Figure 2, there are 207 packets matched to the second part of the filter. These packets have a **HTML protocol with POST or GET method**, and there are just a few milliseconds between the capturing of each packet. It also can be observed that the request line follows a similar pattern: **same host, client IP addresses, and User agents changed constantly**. This can be a sign of a **HTTP flood method**. Different IP addresses indicate that this **DDoS attack** uses a botnet, which uses IP spoofing or leverages addresses of devices infected by malware to flood the server with requests. Constantly changing User agents helps the malicious requests blend in with legitimate ones and bypass certain restrictions. [3]

b. Identifying the Attacker's Host from Wireshark Filter Output

Based on the filtered results, it is suspected that **10.0.0.2** was the attacker's IP address. As shown in Figure 2 provided in section a, **10.0.0.2** was the source that sent all the malicious SYN flags and the http requests.

The attacker can also be identified by examining endpoint statistics. This can be done by navigating to the toolbar, selecting Statistics, then Endpoints, and the Endpoints window will be displayed. As shown in Figure 3, in the IPv4 tab, **10.0.0.2** was the source that transmitted 207 packets. Figure 4 presents the result of the TCP tab; **10.0.0.2** was also the source of several packets each transmitted through a different port. Therefore, it can be determined that **10.0.0.2** was the attacker's IP address.

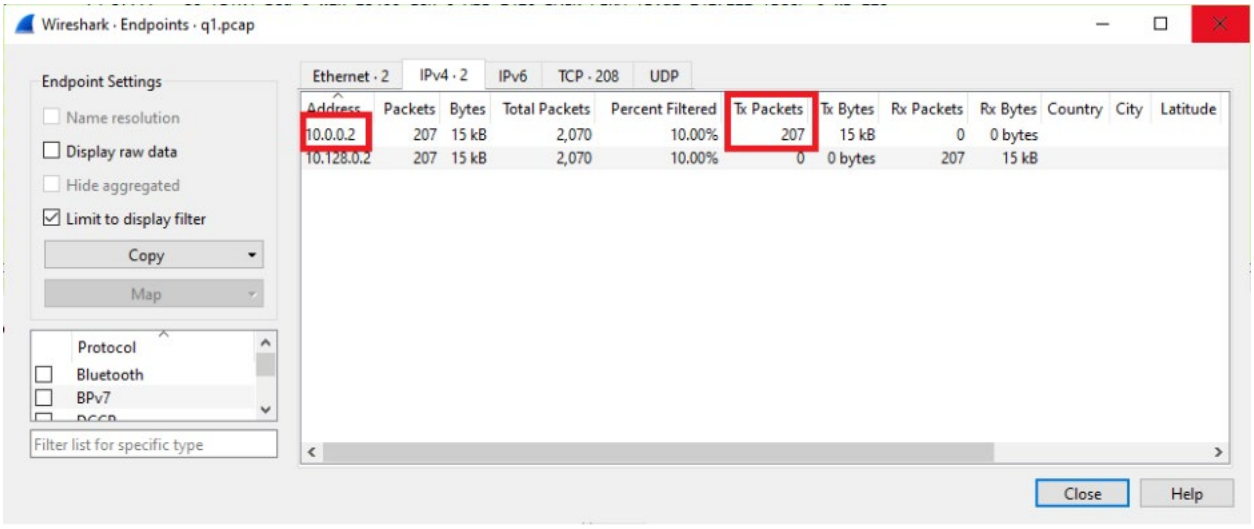


Figure 3: Wireshark Endpoints of IPv4 Transmission

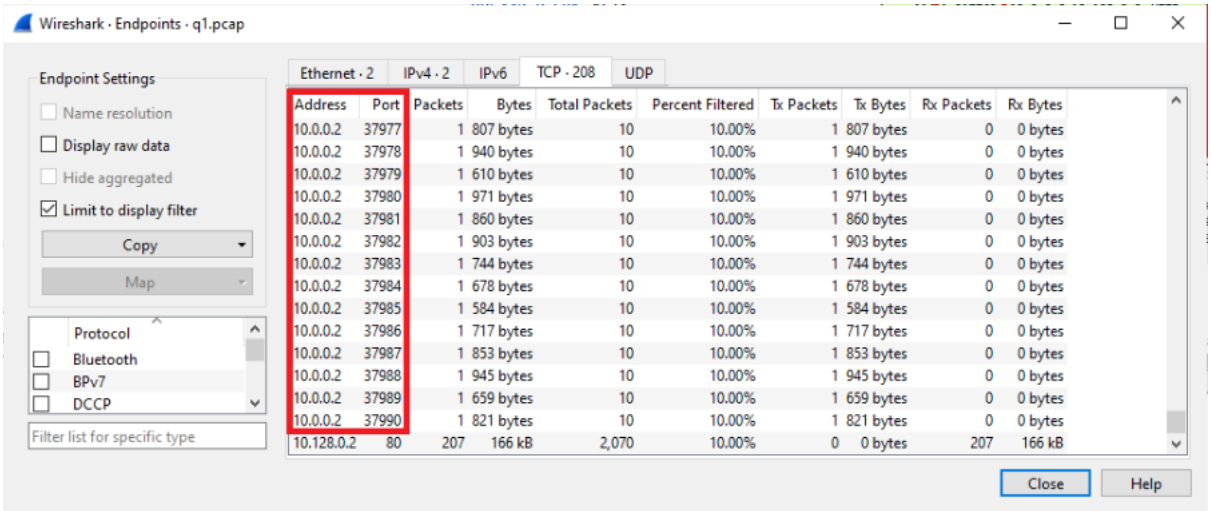


Figure 4: Wireshark Endpoints of TCP Transmission

With the attacker's IP address, a **reverse DNS lookup** can be performed to discover the attacker's hostname.

Q2: Mobile Data Acquisition and Analysis

Part 1:

a. Create an Android Virtual Device using Android Studio

Android Studio is the official Integrated Development Environment (IDE) for Android app development. It provides an environment to develop Android devices and emulators in real-time[4]. In this task, we will create a Virtual Device using Android Studio and practice Mobile Data Acquisition skills and provide analysis based on our findings.

After installing Android Studio, a welcome window like this will be displayed. Select **More Actions** > **Virtual Device Manager** to open the Device Manager window. (As shown in figure 5)

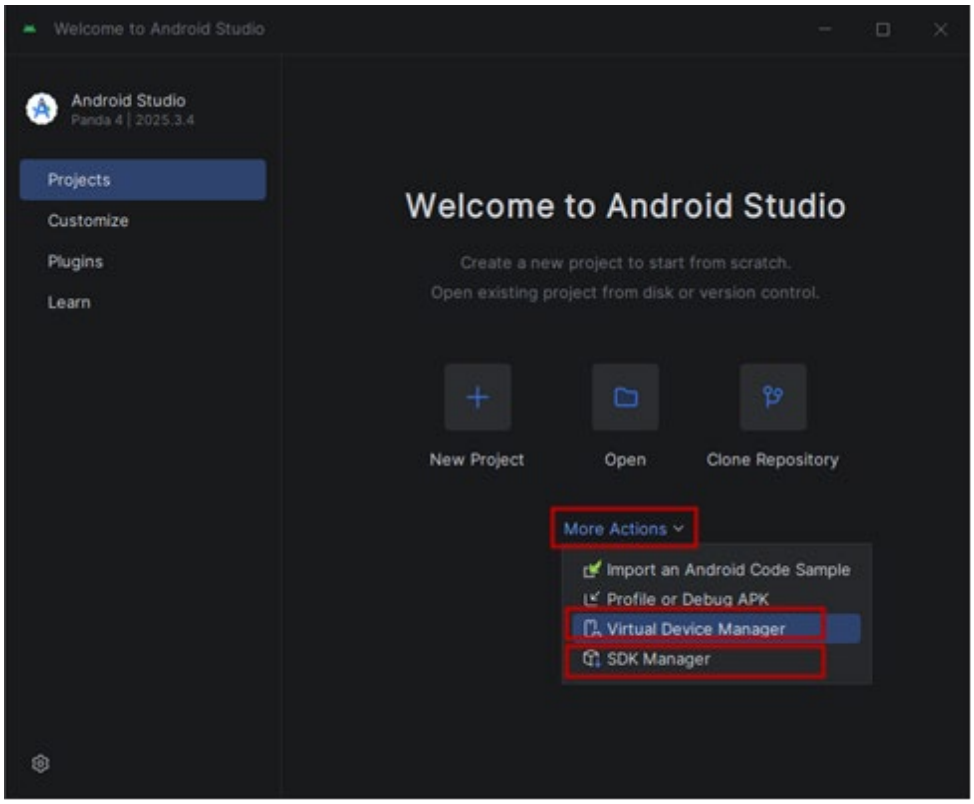


Figure 5: Android Studio Welcome window

This is the Device management window. To start creating an emulator, click on **“Create virtual device…”** in the middle of the window or the **“+”** in the toolbar. Either way, it will open the **Add Device window** (as shown in figure 6).

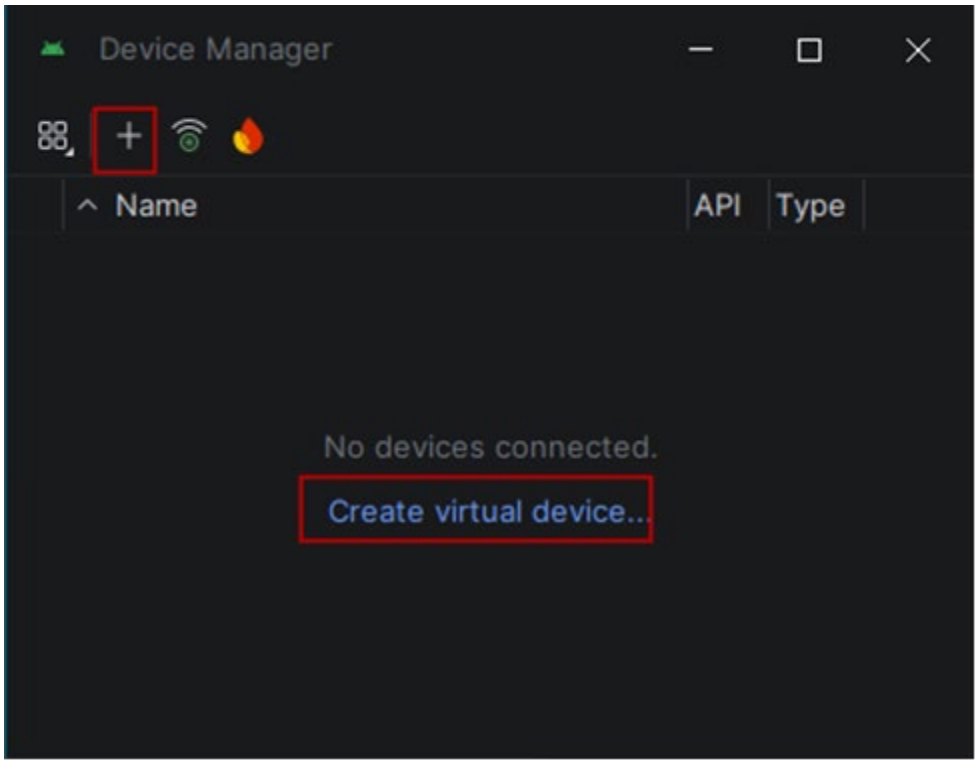


Figure 6: Android Studio Device Manager

For this task, the **Pixel 6 Pro** device will be used as we adopted the instructions from the workshop document. Pixel 6 Pro is an **up-to-date** android device (released in 2021). It also has a **6.7" screen** which will make the visualization of this assignment better. Other devices like **Pixel 4 XL** or **Pixel 7 Pro** can also be used for their large screen, in **consideration of the available resources** of the device on

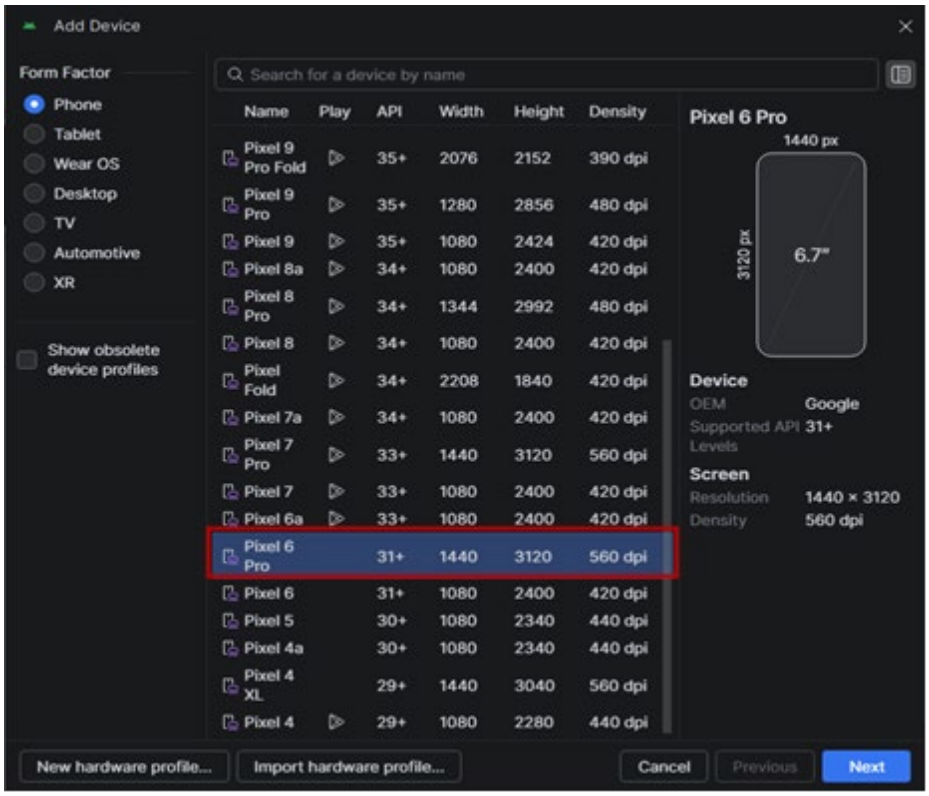


Figure 7: Android Studio Device Selection

which the virtual device will be created (as shown in figure 7).

After choosing the desirable devices, we chose **Android 14 API** and **Google API service. Android 14** is used for its **stability** compared to other latest versions, and **Google API** is chosen because it allows **root access**, which the "Google Play" option will restrict [5]. Installing the system

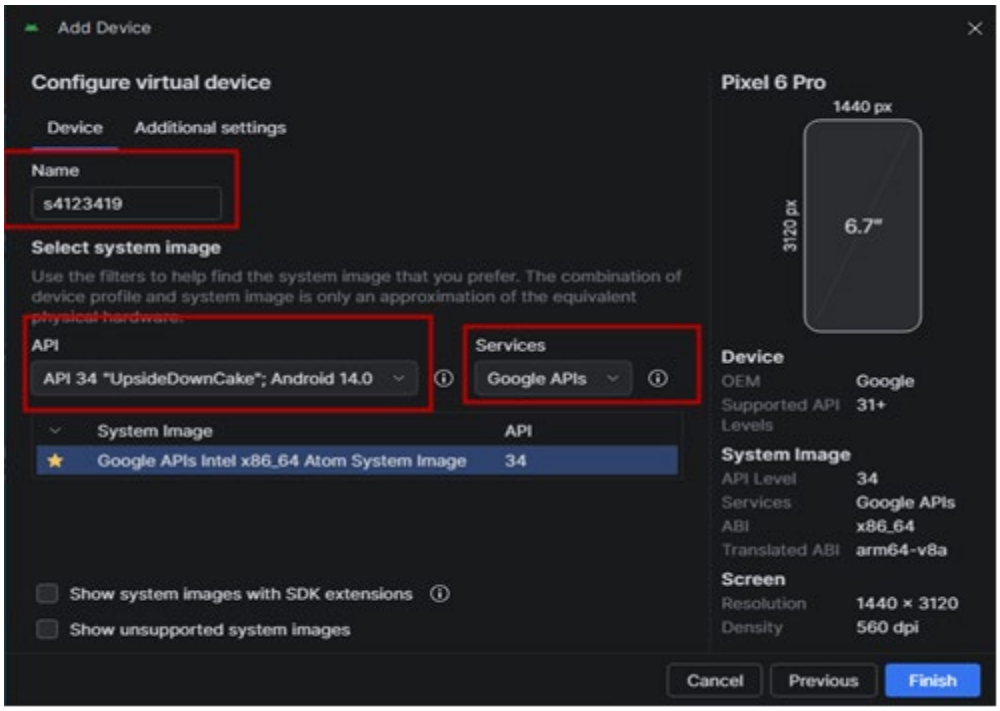


Figure 8: Android Studio Device Configuration

image might be needed before completing this task. The device is named after the student ID of 1 member, as required (as shown in figure 8).

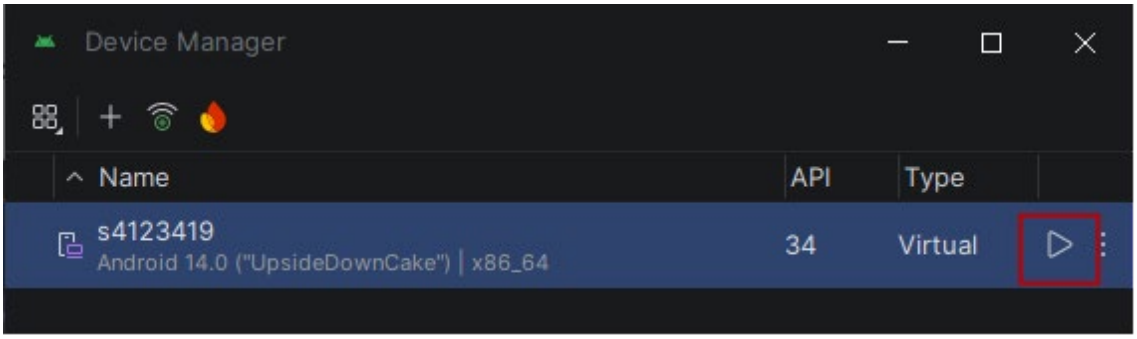


Figure 9: Virtual Device successfully created.

With that being done, the device will be created and shall appear in the device manager. By clicking on the play button **"▷"**, the device is booted (as shown in figure 9).

If successful, a device will be displayed on the desktop (as shown in figure 10).

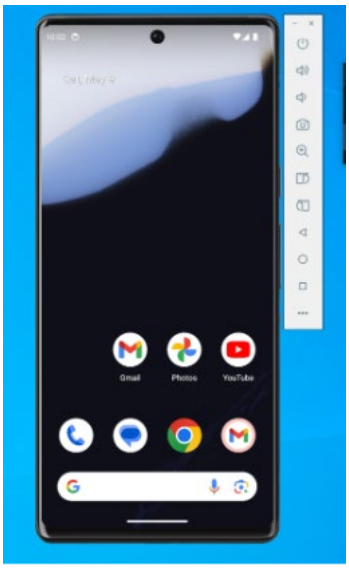


Figure 10: The emulator of Google Pixel 6 Pro

b. Create Contacts and Messages

To create contacts, we used the cursor to try to mimic the finger flick motion to open the **application menu** and select **Contacts**

(As shown in figure 11).

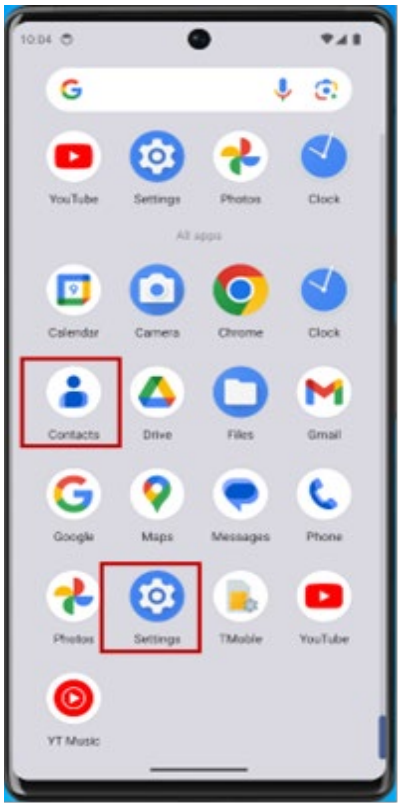


Figure 11:: Pixel 6 Pro application menu

Select the “+” button to create a contact

(as shown in figure 12).

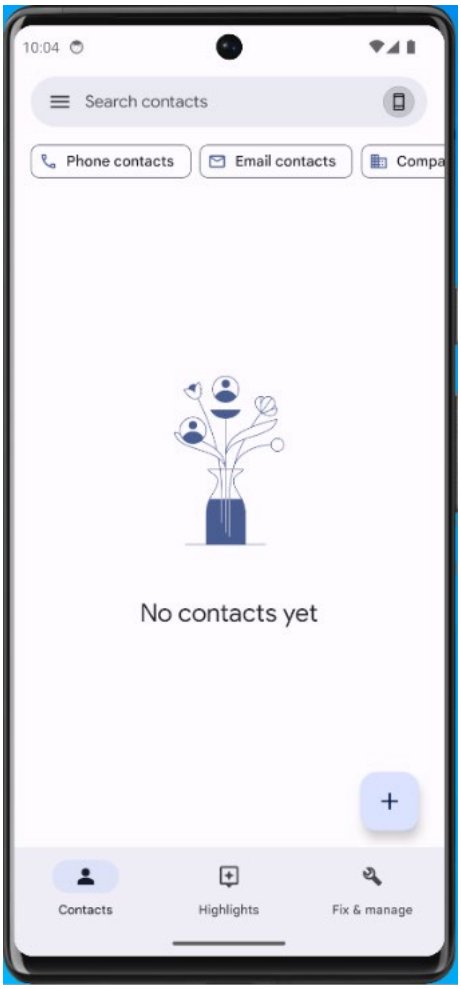


Figure 12: Phone’s Contacts

Here we set the contact's name to one member’s name and set the phone number as **04XXXXXXX1**. Then, we repeat the step with the name of another member and the phone number as **04XXXXXXX2** (with XXXXXXX replaced by our student number

(as shown in figure 13, figure 14).

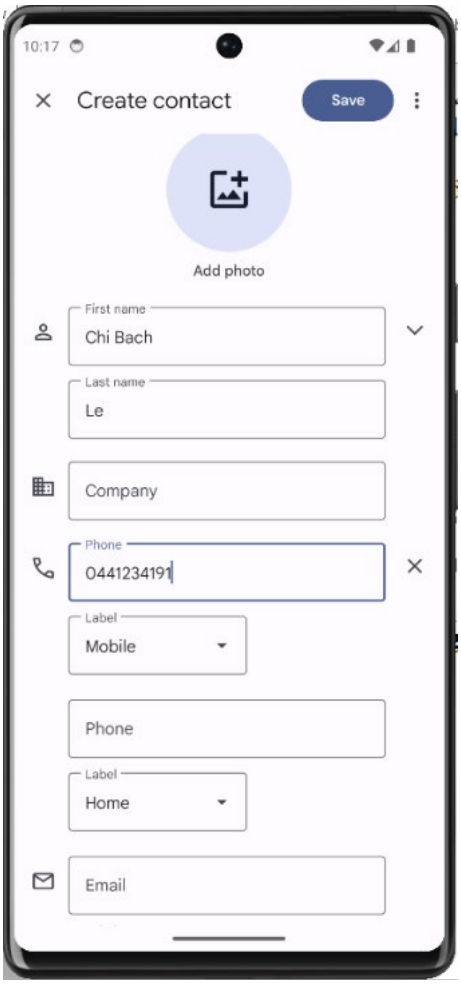


Figure 13: Creating contact

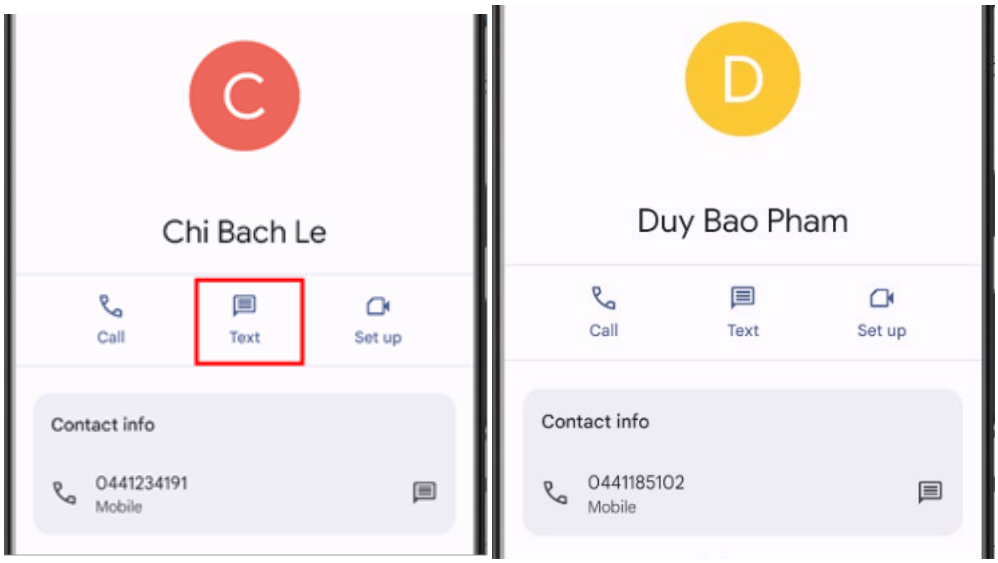


Figure 14: Created contacts

For the next step, we click on the **"Text"** option to be redirected to the Message application and send the messages to the right contact. (As shown in figure 15)

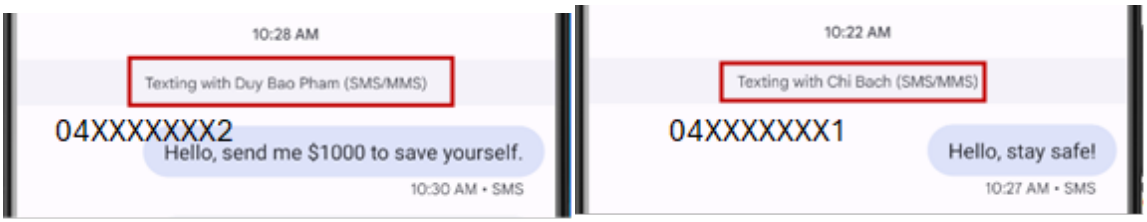


Figure 15: Created SMS

Part 2:

c. Data acquisition using ADB tool

To perform data acquisition on an Android device using ADB tool, the first step is to enter **developer mode** and **enable debugging mode**.

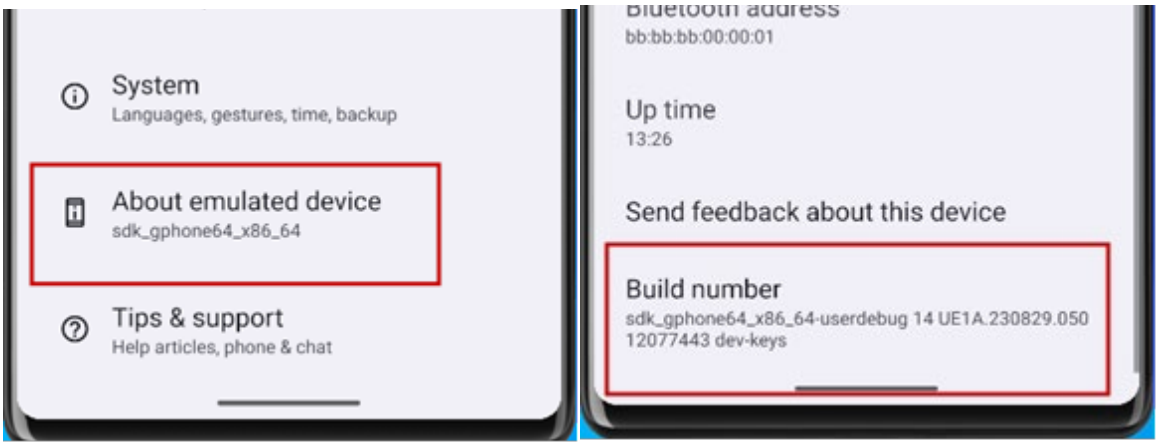


Figure 16: Enabling developer mode

To turn on developer mode, return to application menu (as shown in figure 16), select **Settings** > **About emulated device**. Scroll down and **click Build number 7 times** until there is a message saying: **"You are now a developer"** (because the virtual device was not running smoothly, we were unable to capture the message on the screen).

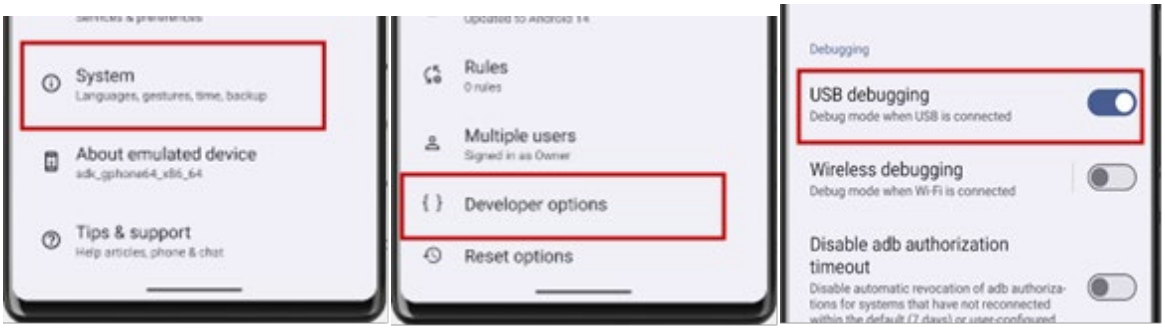


Figure 17: Enabling USB Debugging

Next, we return to **Settings**, select **System** then **Developer options**, and **enable USB Debugging**, which will grant permission for the virtual device to communicate with the host device through **Android Debug Bridge (ADB)** (as shown in figure 17).

ADB is a command-line tool that allows communication with a device. It provides access to a Unix shell that you can use to run a variety of commands on a device [6].

To use Android Debug Bridge, first we need to verify that ADB is installed. ADB is often installed with Android Studio. To verify its installation, we went back to

the **Welcome window** (as shown in figure 23) of Android studio to open the **SDK Manager**.

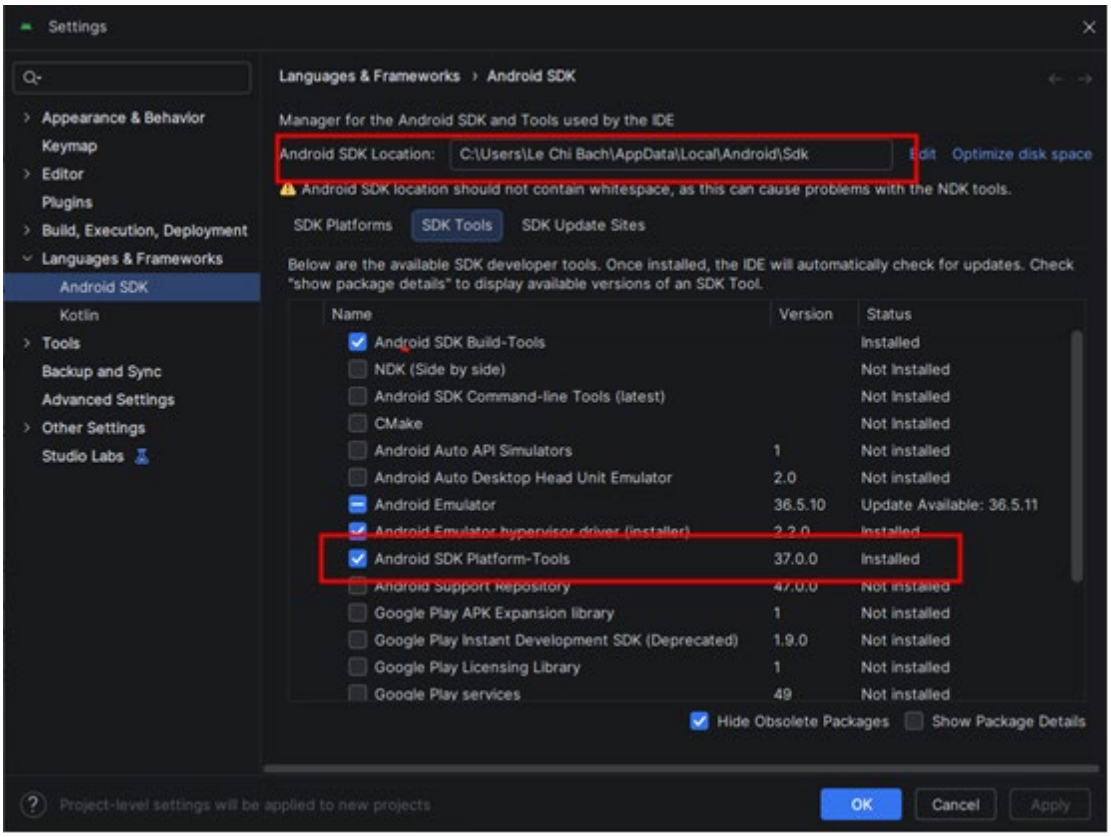


Figure 18: SDK manager

In the list of available tools, it could be seen that **Android SDK Platform-Tools** was checked and labeled as installed. This means that ADB has already been installed. We also took the **SDK location** and added **"\platform-tools"** to access ADB via Windows CLI (As shown in figure 18).

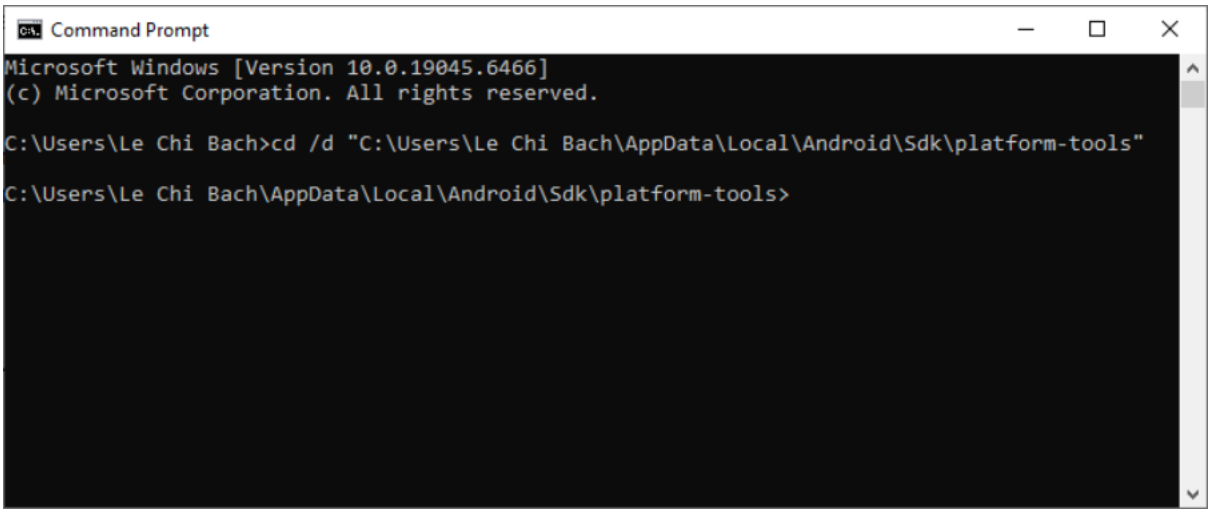


Figure 19: Accessing ADB using Window CLI

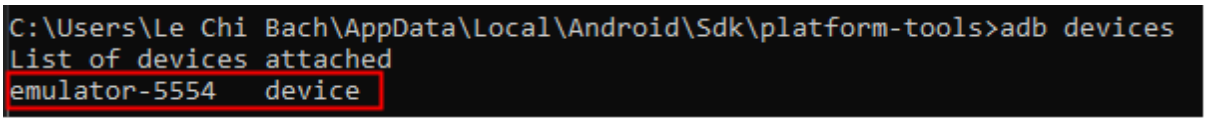


*Figure 20: Running **"adb devices"** command*

We used the **"adb devices"** command to show the **list of attached devices**. It showed one device which was the created emulated phone (As shown in figure 20).



*Figure 21: Running **"adb root"** command*

Next, we run the command **"adb root"**. The returned message was **"restarting adbd as root"**, which meant the **daemon** (the component which runs commands on a device) had been restarted with **root permission** (privileged access) [7]. (As shown in figure 21)

d. Identify the locations and names of the SMS and Contact databases

The SMS database is often named **"mmssms.db"**[8] and the contacts database is often named **"contacts2.db"**[9] The next step was to locate the database on the device. We used **"adb shell"** to access the phone system, then used the **"find /data -name "mmsssms.db"** to locate the exact location of the SMS database. The same is done for **"contacts2.db"**.

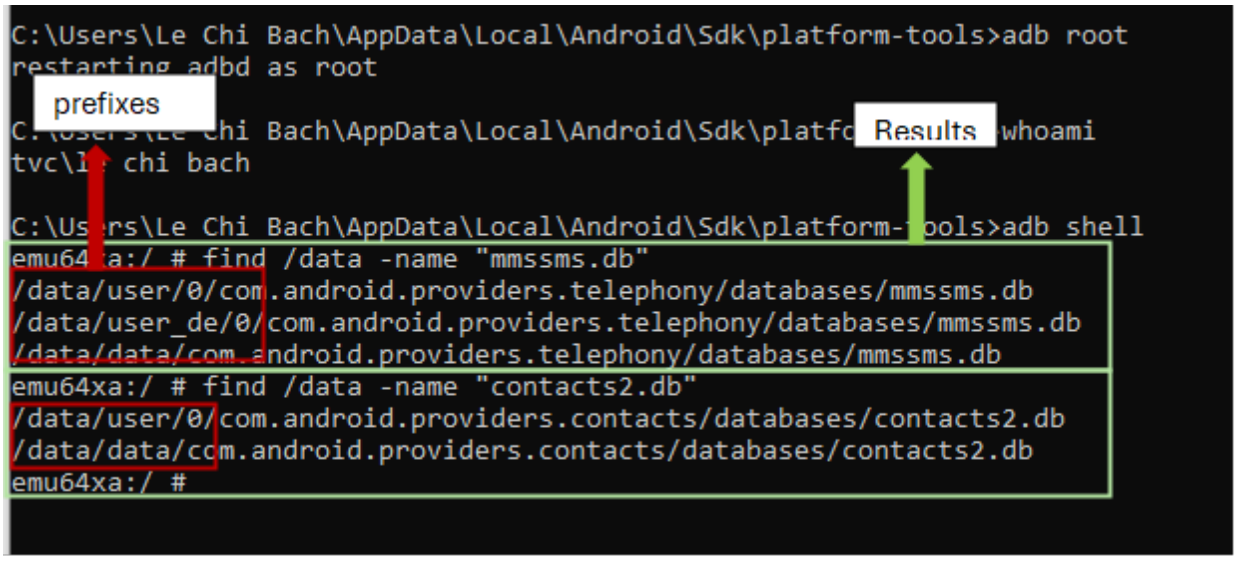


Figure 22: the location of the SMS and Contacts Databases

The results showed 3 results for **“mmssms.db”** and 2 results for **“contacts2.db”** **(**As shown in figure 22). They had different prefixes pointed to different directories. **“data/data”** is where the app databases and settings are stored, and the needed databases will be stored in the **“data”** folder [10]. We expected to find the required databases here. **“data/user/0”** is the system user with the ID of 0 in an android device that allows multiple users. It often belongs to the main user and is a symlink to **“data/data"** [11] means the 2 paths would likely point to the same directory. **“data/user_de/0”** stores data in device encrypted (de) storage, which is the storage location for direct boot mode (and after the user unlocked the device as well) [12]. By default, apps don’t run in direct boot mode. Apps need to be registered to run in this mode. For example, notifications or timers are tasks that run when the device is not unlocked [13]. Messages and contacts list are not the content that can be accessed in direct boot mode (when the device is locked), but they sometimes can still be stored in this partition [12]. Therefore, the databases are not guaranteed to be found in **“data/user_de/0”**. We will attempt to find the databases in the data folder as this is the location where the databases are normally stored [10]

“**data/data/com.android.providers.telephony/databases/mmssms.db”** and **“data/data/com.android.providers.contacts/databases/contacts2.db”** were the commands used to copy the databases to the computer.

e. Copy the databases to the computer.

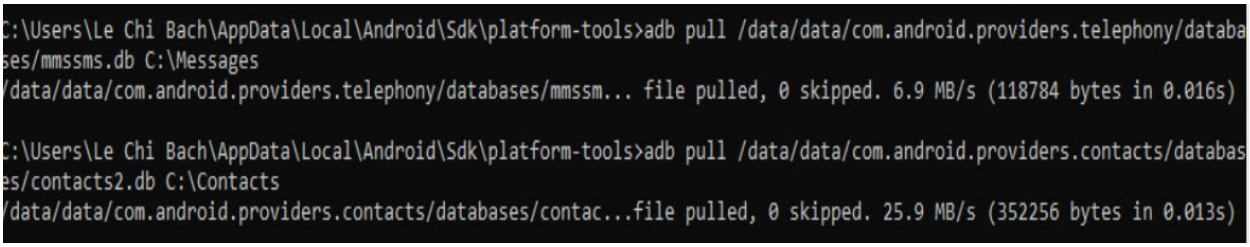


Figure 23: Copying SMS database and Contacts database to the computer

To copy the databases to the computer, we used the **"adb pull [source directory] [destination directory]"** command. The complete command would be

"adb pull data/data/com.android.providers.telephony/databases/mmssms.db C:\Messages" and **"adb pull data/data/com.android.providers.contacts/databases/contacts2.db C:\Contacts"** (As shown in figure 23). After running the 2 commands, we opened the folders specified in the command, and we saw one database in each. (As shown in figure 24 and figure 25)

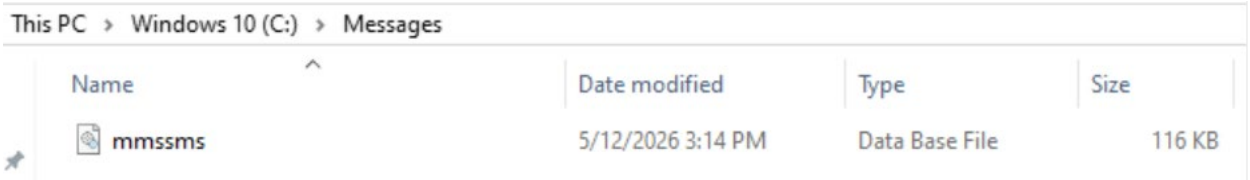


Figure 24: SMS database copied to the computer

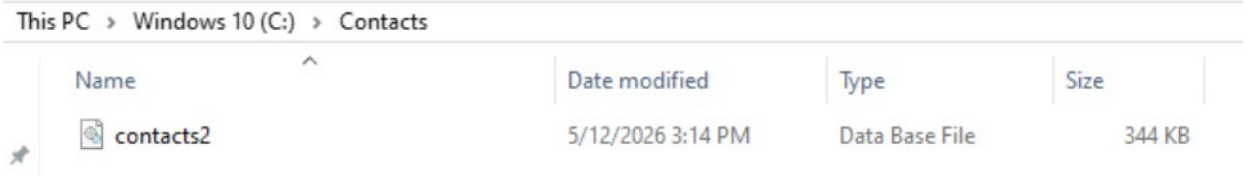


Figure 25: Contacts database copied to the computer

f. Access permission issue

During the implementation of **Question 3-part e**, we did not encounter any **"Access permission"** issue. The reason for that was possibly because we have run the **"adb root",** which allowed running command with high privilege, in **Question**

3-part c. We tried to run the **"adb pull"** command again without root access, and the returned message was **"Permission denied"**. The result (As shown in figure 26) had confirmed our suspicion for the reason why we did not encounter the forementioned problem. In addition, it is important that **Google API** service is chosen in the creation of the device because the other option (**Google Play**) will not provide **elevated privilege**(root) [7].



*Figure 26: Running **"adb pull"** requests without root permission*

g. Retrieve the SMSs and the contact details.

After copying the databases to our computer successfully, we can examine them to see the contacts and messages that we have created in the emulator.

In this task, we used **SQLite Viewer** to read the acquired databases.

After uploading the **mmssms.db**, we found the messages in the **"sms" table**, with the receivers' numbers (As shown in figure 27).

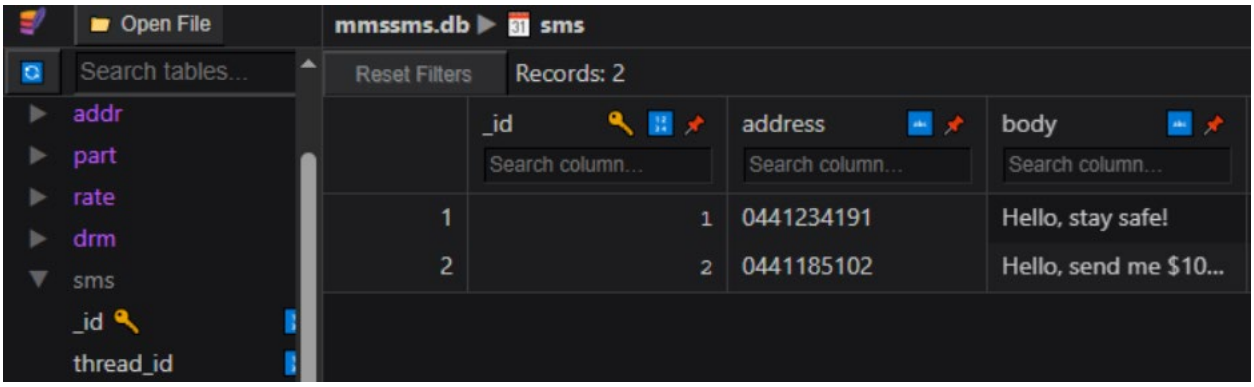


*Figure 27: **mmssms.db** database, **"sms"** table*

The result matched exactly with the contents we created in the emulator.

We also uploaded **contacts2.db** to examine.

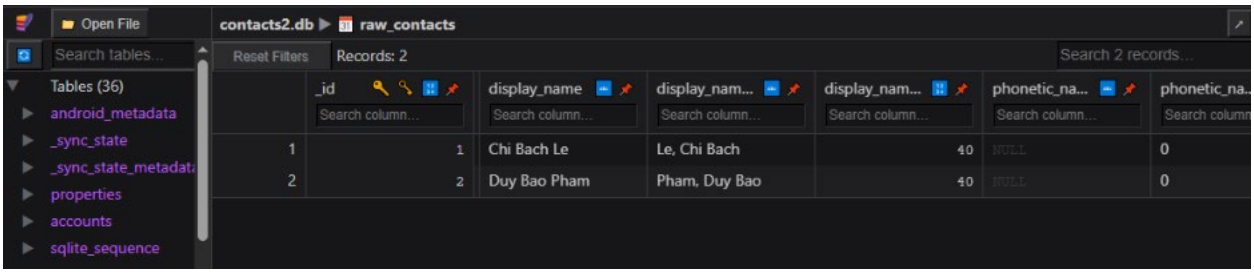


Figure 28: ***contact2.db*** *database,* ***"raw_contacts"*** *table*

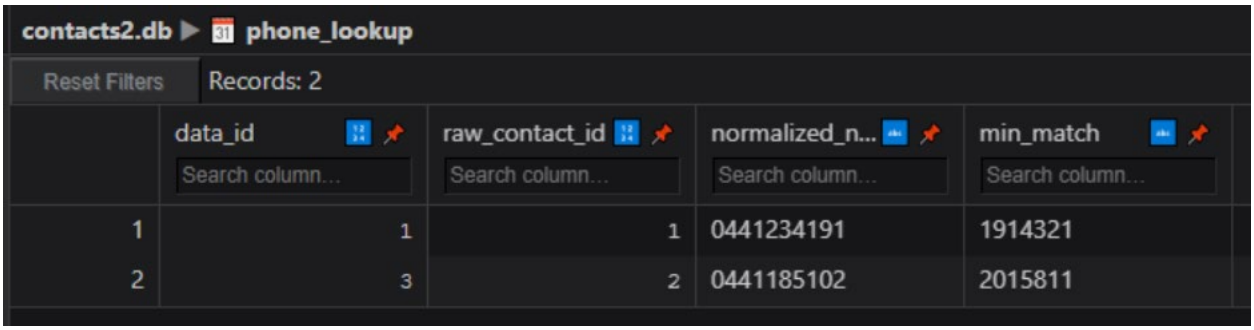


Figure 29: ***contact2.db*** *database,* ***"phone_lookup"*** *table*

In the **contacts2.db**, we found the contact name in the **"raw_contacts"** table, and the phone numbers in the **"phone_lookup table"**, with the **"raw_contact_id"** attribute matching the number with the equivalent **"_id"** attribute in the **"raw_contacts"** table. (As shown in figure 28 and figure 29)

h. Conclusion

Question 3 has demonstrated the methodology of digital forensics on a mobile device, focusing on data acquisition and analysis. In the completion of the task, we have obtained some key insights that we thought would be relevant in a real-world scenario. Extracting artifacts from core system applications strictly requires root access. To perform data acquisition, bypassing production security is a critical step, which is shown in the difference between **Google APIs** and **Google Play** system images. The demonstration of the task also highlights the operation of **File-Based Encryption (FBE)**. The presence of messages and contact databases in the credential encrypted location (**/data/user/0** or **/data/data)** instead of the device's encrypted location confirms that user data privacy is locked until the device is unlocked.

References

[1] GeeksforGeeks, “TCP 3-Way Handshake Process.” Accessed: May 15, 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/tcp-3-way-handshake-process/

[2] Cloudfare, “SYN flood attack.” Accessed: May 15, 2026. [Online]. Available: https://www.cloudflare.com/learning/ddos/syn-flood-ddos-attack/

[3] Murat Alagoz, “Change your User Agent in web pentests and bug bounties: Don’t Be a Plain Tester!” Accessed: May 15, 2026. [Online]. Available: https://mrtalagoz.medium.com/change-your-user-agent-in-web-pentests-bug-bounties-dont-be-a-plain-jane-98b442a0c601

[4] “Meet Android Studio | Android Developers.” Accessed: May 15, 2026. [Online]. Available: https://developer.android.com/studio/intro

[5] “Create and manage virtual devices | Android Studio | Android Developers.” Accessed: May 16, 2026. [Online]. Available: https://developer.android.com/studio/run/managing-avds

[6] “Android Debug Bridge (adb) | Android Studio | Android Developers.” Accessed: May 15, 2026. [Online]. Available: https://developer.android.com/tools/adb

[7] “Rooting (Android) - Wikipedia.” Accessed: May 15, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Rooting_(Android)

[8] “Android Messaging Forensics – SMS/MMS and Beyond - Magnet Forensics.” Accessed: May 15, 2026. [Online]. Available: https://www.magnetforensics.com/blog/android-messaging-forensics-sms-mms-and-beyond/

[9] “Android Contacts.” Accessed: May 15, 2026. [Online]. Available: https://forensafe.com/blogs/Android_Contacts.html

[10] “applications - Where do Android apps store data? - Android Enthusiasts Stack Exchange.” Accessed: May 15, 2026. [Online]. Available:

https://android.stackexchange.com/questions/47924/where-do-android-apps-store-data

[11] "Users for system developers." Accessed: May 15, 2026. [Online]. Available: https://android.googlesource.com/platform/frameworks/base/+/master/core/java/android/os/Users.md

[12] "File-based encryption | Android Open Source Project." Accessed: May 16, 2026. [Online]. Available: https://source.android.com/docs/security/features/encryption/file-based

[13] "Support Direct Boot mode | Security | Android Developers." Accessed: May 15, 2026. [Online]. Available: https://developer.android.com/privacy-and-security/direct-boot